

Department of Engineering Science
Faculty of Engineering

UNIVERSITY OF AUCKLAND

Historical Data Prediction for Geothermal Systems using Data-Driven Modelling

Daniel Clark

Supervisors:

Dr Michael Gravatt
Associate Professor Andreas W. Kempa-Liehr

Partner:

Michael Terekhin

October 3, 2024

Abstract

DO LAST

Acknowledgements

dflojsdfdfzdf

michael

michael, andreas

paul

oliver, ru

uoa

flaties?

Contents

1	Introduction	1
1.1	Problem Background	1
1.2	Available Data	1
1.3	Existing Approaches to the problem	3
1.3.1	Bayesian Modelling	4
1.3.2	Machine Learning Approaches	5
1.3.3	Physics Informed Machine Learning	6
1.3.4	Solving as an Optimisation Problem	6
1.3.5	Inverse Problems	6
1.3.6	Uncertainty Quantification	6
1.4	Research Approach	6
2	Loss Function Optimisation (LFO)	8
2.1	Tools Used	8
2.2	Formulating an Initial Model	8
2.3	Solving LFO Analytically	9
2.4	Relaxing LFO Assumptions	10
2.5	Applying Tikhonov Regularisation to Reduce Overfitting	13
3	Linear Least Squares Optimisation (LLSO)	15
3.1	Reformulating the Problem as an Inverse Problem	15
3.2	Speed Comparison of LFO and LLSO Models	16
3.3	Tikhonov Regularisation for LLSO	17
4	Results with Simulated and Real Data	19
4.1	Quantitative Results of LLSO Model on Simulated and real Data	19
4.2	Quantifying LLSO Model Uncertainty	21
4.3	Comparison with other models	21
5	Discussion	23
5.1	Discussion of LLSO and GPR Model Effectiveness	23
5.2	Conclusions and Future Work	24
6	Bibliography	25
7	Appendices	27

List of Figures

1.1	Geothermal fields and Taupō Volcanic Zone in New Zealand’s North Island. Adapted from [1].	2
1.2	Simplified example of power generation process at geothermal field.	2
1.3	Simplified flowchart of manual process for predicting historical geothermal data.	3
2.1	Well-specific mass flows predicted by LFO model against real results	11
2.2	Mass flows predicted by linear LFO Model against true mass flows.	12
2.3	Mass flows predicted by AHOP LFO model for $k=3$ (cubic) against true mass flows	12
2.4	Mass flows predicted by AHOP LFO model with Ridge Regularisation for $k=3$ (cubic) against true mass flows	14
2.5	Mass flows predicted by AHOP LFO model with Tikhonov Regularisation for $k=3$ (cubic) against true mass flows	14
3.1	Equations $W_i(t)$ for each well, when solved for $k=2$ by LFO and LLSO Models	16
4.1	Mass flows predicted by LLSO model for simulated data, for $k = 4$ and optimal parameter values.	20
4.2	Mass flows predicted by LLSO model for real data, for $k = 4$ and optimal parameter values.	20
4.3	25th to 75th percentiles of mass flows predicted by LLSO model for real data, for $k = 4$ and optimal parameter values.	22
4.4	95% confidence intervals of Michael’s GPR Model for real data.	22

List of Tables

3.1 Speedup of new method compared to old method for different T and k
values, with fixed n=5, rounded to nearest 100%. 17

Chapter 1

Introduction

1.1 Problem Background

Geothermal energy is a renewable and environmentally-friendly energy source which can last for thousands of years [2]. Geothermal reservoirs hold water (in liquid and gaseous forms) at high temperature and pressure in large underground volumes of porous rock. This hot steam can be extracted to provide physical heat to a town or wider area, or it can be used to generate electrical energy by driving large steam turbines [3]. The electricity generated by this can then be fed into the electrical grid, which both provides the country with a clean source of renewable electricity generation, and can earn profit for whoever is running the geothermal site.

New Zealand’s North Island is fortunate to have an abundance of Geothermal reservoirs. The majority of these lie in the Taupō Volcanic Zone, which runs Southwest from Whakaari (White Island) through Lake Taupō, to Mount Ruapehu, as shown in Figure 1.1. All of New Zealand’s available geothermal resources have been identified, and we are not able to create any more, so we must be sure that we are using these geothermal resources responsibly. Because of the numerous benefits of using geothermal electricity generation, it is important to invest in geothermal energy, and ensure that we are properly caring for our geothermal resources. If too much mass is extracted from a geothermal well, then it may drastically reduce the amount of time the geothermal well can operate for. It is important to not extract too much mass from a geothermal reservoir to maximise its active lifespan [4]. To determine whether we are taking ‘too much’, we can use reservoir models to monitor the health of our geothermal reservoirs. Geothermal reservoir modelling is also used to predict the future of geothermal fields. It is important that these models are accurate because the models created can be used to make important decisions regarding the continued operation of the wells, and can act as strong evidence of profitability, driving economic investment in the field.

1.2 Available Data

The effectiveness of a reservoir model depends highly on the accuracy and quantity of the measured data that it is provided. Dense, accurate data with a high certainty-level is required to make helpful predictions, however there is often limited data available. Often, the only frequently measured data is ‘total steam flow’ which is measured at a separator. Separators are normally fed by multiple wells, so it is often unclear how much of this steam has come from each of the wells that contribute to the separator. It is important to isolate how much mass has been extracted from each individual well. Figure 1.2 shows that these measurements taken at the generator or separator are not representative of a single well, but rather of a number of wells, which are each contributing an unknown amount of mass to the separator. These wells will extract both liquid water and gaseous vapour from underground reservoirs, which are separated in a separator. The liquid water component of the extracted mass is generally re-injected as ‘brine’ into another well, while

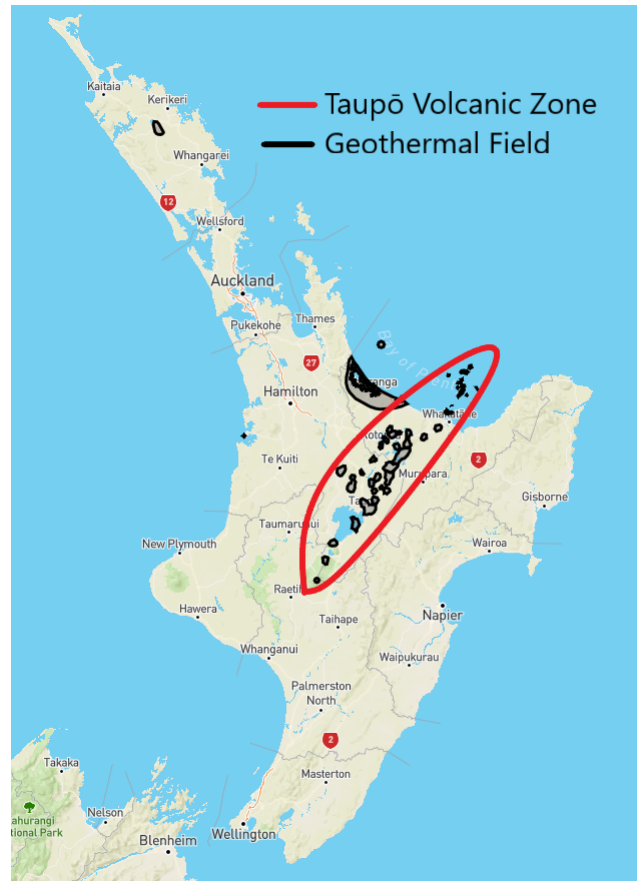


Figure 1.1: Geothermal fields and Taupō Volcanic Zone in New Zealand's North Island. Adapted from [1].

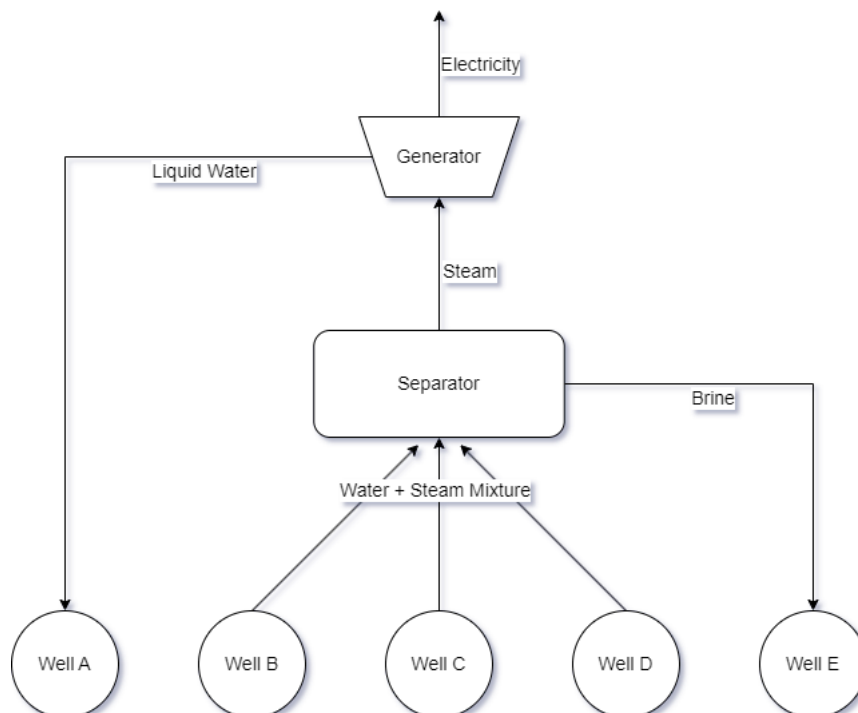


Figure 1.2: Simplified example of power generation process at geothermal field.

the extracted steam continues to the generator, where it generates electricity.

To determine how much mass is being extracted from an individual well, as required for effective reservoir modelling, we must use measurements that were taken at the well itself. These are called Total Flow Test (TFT) measurements, and are only taken occasionally for any given well, as the process of taking these measurements is difficult, and may require turning the well off for a short time. Because TFT measurements are taken so occasionally, often the available data for reservoir modelling is very limited. Using a combination of TFT measurements and total steam flow measurements, it is possible to predict the extraction history of a given well, however it is difficult and time-consuming to do this manually.

Manually inferring dense data from sparse measurements is an iterative process that follows the general structure of an educated ‘guess and check’. It begins by estimating the dryness and average contribution of each well that is contributing to the separator, by inspecting the relevant TFT measurements. Next, the total steam flow that would result from these estimates is calculated, and compared to the measured data. This allows the initial guess to be modified, aiming to change the estimate to meet the data, repeating until a subjective level of ‘closeness’ has been reached. This is not an efficient or accurate method. This method assumes that values such as dryness and well contributions remain constant over time, which is not a realistic assumption. Uncertainty of the produced data cannot be quantified, and the subjectivity of the process leads to added variability in results. There is a need to improve this process by automating the historical data prediction, which is why this project was created.

Myself and Michael Terekhin, my project partner, will use data-driven modelling methods to automate this process. Alongside supervision from Michael Gravatt, a geothermal reservoir modelling expert and Andreas Kempa-Liehr, who is an expert in machine learning and Bayesian modelling, we will model this situation in a number of ways, moving forward with the methods that prove most useful. This report will begin by noting relevant research from related works, and investigating which data-driven modeling approaches could work best for our unique situation. Then we will outline our process, highlighting methods which work poorly or well, and show how we iteratively improve our model before presenting our final data-driven model(s).

1.3 Existing Approaches to the problem

Geothermal reservoir modelling is a large field where many approaches have been used to model geothermal wells. There are three classifications of geothermal reservoirs: high temperature above 225 degrees Celsius, low temperature below 125 degrees Celsius, and intermediate temperature between [5]. Higher reservoir temperatures lead to higher enthalpy of extracted fluid, and mean more electrical energy can be generated. This why it is important to use reservoir modelling to model the pressure and temperature of a geothermal reservoir over time. It gives the ability to predict the future temperature and pressure of a given reservoir, so that business decisions can be made to maximise profitability. Reservoir modelling can also ensure that we do not extract too much liquid from the reservoir, so as to keep the reservoir stable and healthy. Reservoir modelling

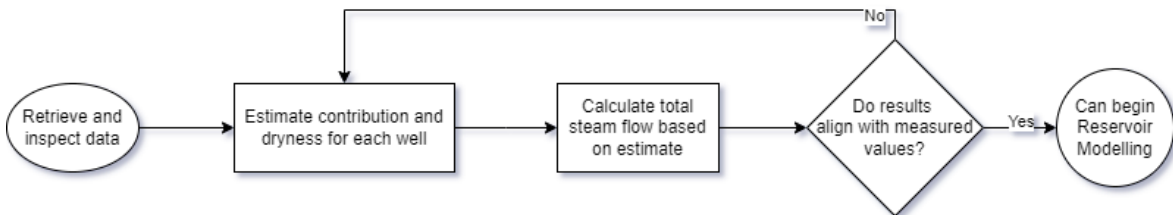


Figure 1.3: Simplified flowchart of manual process for predicting historical geothermal data.

generally models only a single well/reservoir within a larger geothermal field, so this is why it is important to predict the geothermal history of individual wells from total steam flow data, as we are doing. Below, we will discuss a number of techniques that could be used for predicting the required data, and discuss the advantages these techniques can provide. In Methodologies, we will begin using these methods to generate data-driven models.

1.3.1 Bayesian Modelling

Bayesian Modelling is a statistical method where prior distributions based on existing contextual knowledge are used to generate posterior distributions, which can be used to generate predictions and make analyses [6]. While Bayesian modelling has been applied to a geothermal context before [7], it has not been directly applied to the data prediction scenario described above. The most similar existing research largely comes from unrelated fields, where a similar allocation problem was encountered, such as the ‘Bellybutton Problem’, and the ‘Lions, Tigers and Bears’ discussed in the book ‘Think Bayes’ [8]. This suggests that, when undertaking Bayesian modelling of these situation, when it is known that all variables must sum to one, that a Dirichlet distribution is the appropriate prior distribution.

One approach within the Bayesian framework is Monte Carlo simulation. This is a method involving repeatedly generating random samples taken from the prior distribution, and then selecting the samples that go ‘near’ our data points, as feasible solutions, which are then used to generate the posterior distributions [9]. This can help us to quantify the uncertainty [9] of our estimates through results such as prediction intervals. However, this can lead to overfitting of uncertain and potentially noisy data. To prevent overfitting, we can use ensemble methods. These are methods such as Bayesian Voting, that have many models simultaneously and then take a weighted average of all the models’ results to give a final result [10]. This method of combining multiple ‘weak’ regressors into one ensemble model helps to reduce overfitting, because each individual model is trained on only a subset of the training data. This means they are less fit (they may even be underfitting) as individuals, but as a whole come together to give good results.

When engaging in traditional Bayesian modelling and computation, it is important to know the relevant likelihood function or at least be able to estimate it by computing it numerically. In some situations this is not available and this is when Approximate Bayesian Computation (ABC) can be used to sidestep the likelihood function [11, 12]. ABC is a likelihood free inference method where samples are generated from the prior distribution, and run through the model to examine their similarity to the posterior distribution, using summary statistics, such as the mean, variance, range, maximum [13], although the types of summary statistics that can be used are highly context-dependent. Using a metric such as Euclidean distance, we measure if these statistics are ‘close’ enough to those of the posterior, and if so, accept that sample. As this is an ensemble method [12], this is then repeated to generate many samples. This makes it resistant to overfitting so could be a good method to use. The choice of summary statistics is important and often requires domain knowledge to choose appropriate statistics [14], however the process can be more computationally efficient than other Bayesian methods due to the summary statistics reducing the dimensionality of our model.

Gaussian Process Regression (GPR) is a Bayesian machine learning method that involves using a kernel to define a covariance matrix between variables. GPR can only be used when certain assumptions are met, including that the dataset, and the associated noise within, follow Gaussian distributions [15]. While our dataset cannot have perfectly Gaussian distributions, as the physical requirement that all contributions at a given time sum to one ensures that samples are not independent. Also the lower bound of 0 and upper bound of 1 ensure that a true Gaussian distribution is not appropriate, however, the Dirichlet distribution as mentioned above acts locally in a similar way to a Gaussian Distribution, so it is worth investigating whether this method can be applied to our dataset.

1.3.2 Machine Learning Approaches

Machine learning (ML) is the process of creating algorithms that learn how to improve themselves from their experiences [16]. In practice, a machine learning model is a model that will get better with the more data you train it on. There are many types of learning algorithms, some of which we will discuss below. The general goal of a machine learning algorithm is to find a model that minimises a chosen loss function [17]. When each additional piece of training data is fed into the algorithm, it will work out how to use the additional information to keep minimising the loss function, getting better and better until it runs out of training data. However, this process can lead to overfitting, which is when a model learns too effectively on the training data to the point that it captures the noise within the training data, instead of just capturing the desired real trend. To avoid overfitting, we often add a regularisation term onto our loss function.

There are many different types of regression but important ones which I will consider are Lasso regression, which aims to decrease the number of non-zero parameters in our model and performs feature shrinkage, Ridge regression, which decreases the magnitude of parameters in our model, and Tikhonov regularisation which aims to 'smooth' out a model by minimising its first, second or higher order derivative(s).

Following the lead of the oil and gas industries, the geothermal industry's use of machine learning has been increasingly exponentially since 2018 [18]. In a geothermal context it has previously been used in the context of well exploration, heat flux prediction, reservoir modelling, and above-ground component modelling, but not been used previously in the exact context we are looking to use it in [19]. We believe that it could be effectively applied in this area to automate the historical data prediction process.

Random forest regressors use an ensemble of decision tree regressors to predict the desired output. The decision trees each involve a sequence of splits from the root node down to the leaf nodes, where the leaf node will give the input data a predicted output. Each of these decision tree regressors can be given a maximum depth and complexity, and the large number of trees effectively being 'averaged' make it very difficult to overfit a random forest model [20]. Random forests also require relatively few computations, therefore are a very appealing option for early machine learning models.

Neural Network machine learning models often need to be provided large amounts of training data [21] to become effective predictive models. This may be an issue in our context because the amount of data we could provide it is limited by the data which has been measured at the geothermal site. Keeping this in mind, this limits the sorts of models which we will attempt to use. For example, deep learning methods often require large amounts of data to become more effective than simpler models [22].

However, we may be able to sidestep this issue by generating additional simulated training (and testing) data to feed our models. Gravatt and O'Sullivan have created existing simulations that could generate the data required. Also, from one simulation, we can exclude and include different pieces of data to effectively create multiple pieces of training data from just the one simulation. So while we may not be able to feed much data into our model to generate predictions, our model will have a lot of data to learn from, so could still generate helpful results. If we use a (deep) neural network we should be able to extract all possible information from the limited data provided and has a chance to model our situation effectively. We can also use data augmentation to generate additional pieces of data [21] if our NN still did not have enough data to give effective results. Data augmentation involves creating copies of training data that are similar, but have been modified in some realistic way. For example, in our context, to augment training data, we could choose a different subset of TFT measurements to be provided to our model, such that it appears as a completely new piece of training data from the model's perspective. We could also augment data by artificially adding noise (small perturbations) to our dataset.

1.3.3 Physics Informed Machine Learning

The issue with the above machine learning methods is that the contribution outputs for each well are not guaranteed to sum to one. This means that while our predictions may look accurate, they will be physically impossible. To fix this issue, we will have to add physics-informed constraints to our models, to ensure that physical laws are still being obeyed. For example, we must ensure that all contributions sum to one, we expect that the contributions will change smoothly over time, and that the dryness fraction will always be within the expected physical bounds of 0 and 1.

One way to do this is by using a Physics-Informed Neural Network (PINN). PINNs are able to incorporate these physical laws into their loss functions, where multiple functions can be minimised at once. The final loss function can become a sum of many individual loss functions relating to different physical laws, which are summed together with chosen weights which relate to the relative importance of those laws. Upon iteration through the network, the loss function will be minimised, thus specifying that the physical laws are being followed as they should. There are also loss functions applied for the accuracy of the data, and for the initial conditions and boundary conditions being met. PINNs generally work in discretised parameter spaces, which is appropriate to use in our context, because the data taken has been taken at discrete moments in time.

Another benefit of using a PINN is that it can model any physical scenario, as long as it has an accompanying differential equation. There is no rule on how 'complicated' the equation can be and it is not required to have an analytical solution to be applied to a PINN. As geothermal physics can be represented by a highly complicated partial differential equation (PDE), this is ideal. It has also been used to model solutions in the field of fluid mechanics, and used on the notoriously difficult Navier-Stokes equations [23]. ML-augmented simulation has also been used by ANSYS, a premier fluid simulation tool, to "speed up the simulation by factors of 100x", according to an ANSYS blog [24].

1.3.4 Solving as an Optimisation Problem

Another possible approach that could be taken is to solve this as an optimisation problem. As mentioned earlier, a Loss Function is a measure of the error between a predicted solution and a set of measured data. If we can design a loss function to represent our system as described in section ??, then we can take the gradient of this loss function and set it to zero to find the analytical minimal of our system

1.3.5 Inverse Problems

Mention stuff from Ollie (and Ru?)

1.3.6 Uncertainty Quantification

mention uncertainty quantification

1.4 Research Approach

The scope of this research involves the process of building a computational model to effectively infer the historical data history of a set of wells in a geothermal field. This data should be suitable to provide to a predictive geothermal model, and should be accompanied by the appropriately quantified uncertainty of the produced data]. The models using the data that we produce are out of scope for this project. The physical scope pertains to a geothermal field and its associated wells (real or simulated), which will provide a realistic amount of sparse data to our model. The models may be trained on data from many geothermal fields, but will model results for only one section of a geothermal field which feeds into a single separator (as shown in Figure 1.2) at a time. It would be possible in future to expand the scope of the model to include more complex geothermal systems which have components interlinked in more complicated ways.

Myself and my project partner Michael Terekhin will each attempt to build a model that effectively solves this problem. Personally, I will attempt to use numerical optimisation to find the theoretically optimal solution for the historical data prediction. This will begin with a naive model which assumes a constant extraction from each well and over time relax this assumption to build a model that has more flexibility and can represent the real results better. Then, other forms of complexity will be introduced, such as Tikhonov regularisation and parameter tuning, to improve the output of my model on the provided simulated data set. [I will also conduct uncertainty quantification for my model, to ensure that those using the data generated can understand and use the variance of my predictions, so that they can correctly calculate the variance of their own predictions.]

Concurrently, my project partner Michael Terekhin will be attempting to solve the same problem using Gaussian Process Regression. He will also start with a simpler model, and slowly build its complexity until it can fully account for the processes occurring in a given geothermal system. Once Michael and I have both created our models, the results of our respective models will be compared to see which of the models is more effective under different situations. For example, we will test each of them under both low-data and high-data scenarios. We will also investigate whether a joint model that combines results from both of our models would be superior to either individual model. Chapters 2 and 3 explore how the models were created through an iterative process of starting with a simplified version of the situation, and then adding complexity to get a better model over time.

Chapter 2

Loss Function Optimisation (LFO)

2.1 Tools Used

A number of Python packages will be used to develop the models explained in this section. Sympy, Numpy, Scipy, Matplotlib, Pandas and Time are all used. Furthermore, Jupyter notebook will be used as the main environment to develop and tune the model. Jupyter notebook is a web tool for using interactive code documents. Code documents can include Python code, equations, plots, Markdown text and more. It is preferable to a traditional coding environment such as VS Code or Pycharm because it has the ability to run only certain code chunks at a time, as opposed to traditional programming scripts which run the entire program. This reduces the computation cost of developing the model, and also makes it simpler to visualise the model output in the form of plots nested in between code chunks.

ARDC Nectar [Research Cloud](#) is an Australian cloud computing service specifically designed to enable large-scale computing research for academics across Australia and New Zealand. It has a Jupyter Notebook service that allows Notebooks to be run remotely, which was used to run the majority of code during this project. The existing sci-py environment is the one used for this project, as this included Sympy, Matplotlib, Numpy, and all other packages that were required.

cite

2.2 Formulating an Initial Model

Initially, a simplified version of the problem will be modeled where all wells have a constant average extraction rate. $w_{i,t} = w_i$, turning w from a 2 dimensional matrix as shown in equation (??) into a 1 dimensional vector. It is also assumed that simulated data provided is consistent and that measurements can be treated with 100% certainty. Over time, these assumptions will be relaxed, and add additional complexity to better capture the behaviour of the system.

A loss function is a function which takes in a set of decision variables, and evaluates how well these map to data by evaluating the associated 'cost' or 'error'. Different machine learning algorithms use different methods to minimise the loss function.

To formulate the loss function, the squared error measured against each of the two data sources is summed. Equation (2.2.1) shows the total squared error of the sum of all predicted mass flows measured against the total measured mass flow. Similarly, equation (2.2.2) shows the squared error of our predicted mass flow rates against the known TFT values, for all timepoints when this is known. These equations assume a constant average mass extraction rate, and known dryness throughout the system.

$$S1 = \sum_{t=1}^T (M_t - \sum_{i=1}^n z_{i,t} w_i)^2 \quad (2.2.1)$$

$$S2 = \sum_{i=1}^n \sum_{t=1}^T d_{i,t} (m_{i,t} - w_i)^2 \quad (2.2.2)$$

where:

w_i = Average extraction mass flow rate of well i
 M_t = Total extraction mass flow rate recorded between all wells at time t
 $m_{i,t}$ = Extraction mass flow rate of well i at time t
 $z_{i,t}$ = 1 if well i was on at time t, 0 otherwise.
 $d_{i,t}$ = 1 if well i's flow rate was measured at time t, 0 otherwise.
 T = Total number of discrete timepoints
 n = Total number of wells in our system

These two expressions for the squared error measured against our two known sets of data can then be summed together, and weighted with a regularisation parameter, in the form of equation (2.2.3). This gives us the final expression for our initial loss function, shown in (2.2.4).

$$\mathcal{L}(w_1, w_2 \dots w_n) = S1 + \lambda S2 \quad (2.2.3)$$

where:

$\mathcal{L}(w_1, w_2 \dots w_n)$ = Total error value of loss function
 λ = Regularisation parameter that controls the relative weight of error against our bulk and TFT measurements.

$$\mathcal{L}(w_1, w_2 \dots w_n) = \sum_{t=1}^T (M_t - \sum_{i=1}^n z_{i,t} w_i)^2 + \lambda \sum_{i=1}^n \sum_{t=1}^T d_{i,t} (m_{i,t} - w_i)^2 \quad (2.2.4)$$

2.3 Solving LFO Analytically

Initially, equation (2.2.4) was solved by using a built-in Python SciPy minimisation solver, which did give good results but took an extremely long time to converge. It was clear that expanding the problem in any way would push this method of solving the problem into infeasibility. To fix this, a new method of solving for the minimum of equation (2.2.4) was devised. Using the Sympy package, it is possible to analytically differentiate this equation against each decision variable w_i and set these to 0 simultaneously, as shown in equation (2.3.1).

$$\frac{\partial \mathcal{L}(w_1, w_2 \dots w_n)}{\partial w_i} = 0 \quad \forall i \in [1, n] \quad (2.3.1)$$

Because our original Loss function deals with squared errors, these derivatives are at most linear in each w_i . This means that by taking all n of these equations, subtracting the constant terms to the right hand side, and combining the n equations into matrix form, the relationship shown in (2.3.2) is produced. Therefore the optimal w vector can be found by evaluating this expression, either by evaluating $A^{-1}b$ or other equivalent technique built into Sympy's solve() function.

$$Aw = b \quad (2.3.2)$$

where:

w = n-long vector of optimal w_i values

$A \in \mathbb{R}^{n \times n}$
 $A_{i,j} = j\text{'th coefficient of } \frac{\partial \mathcal{L}(w_1, w_2 \dots w_n)}{\partial w_i}$
 $b \in \mathbb{R}^n$
 $b_i = \text{constant term of } -\frac{\partial \mathcal{L}(w_1, w_2 \dots w_n)}{\partial w_i}$

In equation (2.2.4) the parameter λ represents a weighting of the importance given to the sum squared error against the TFT measurements relative to the importance given to the sum squared error against the bulk measurements. To find an appropriate value, it can be observed that the error within the bulk term will be approximately n times that of the error within an individual well measurements, because bulk values are n times larger, as they are made from the contributions of n wells. Building on this observation, equations (2.3.3) and (2.3.4) show how the 'balanced' value for λ can be derived, which will give an approximately equal error contribution from both the bulk measurements term (equation (2.2.1)) and the TFT measurements term (equation (2.2.2)).

$$\begin{aligned}
E_{Bulk} &\approx n \times E_{TFT} \\
(E_{Bulk})^2 &\approx n^2 \times (E_{TFT})^2 \\
\sum_{t=1}^T \sum_{i=1}^n d_{i,t} (E_{TFT})^2 &\approx D (E_{TFT})^2 \\
\sum_{t=1}^T (E_{Bulk})^2 &\approx T (E_{Bulk})^2
\end{aligned} \tag{2.3.3}$$

where:

E_{Bulk} = the average error for the bulk measurement term
 E_{TFT} = the average error for the TFT measurement term
 D = the amount of TFT measurements provided to the model

Substituting results of equation (2.3.3) into (2.2.4) gives equation (2.3.4), which can be solved to neatly express λ for a 'balanced' weighting between TFT and bulk measurement, λ_b .

$$\begin{aligned}
\mathcal{L} &= \lambda_b D (E_{TFT})^2 + T (E_{Bulk})^2 \\
\mathcal{L} &= \lambda_b D (E_{TFT})^2 + T n^2 (E_{TFT})^2 \\
\lambda_b D (E_{TFT})^2 &\approx T n^2 (E_{TFT})^2 \\
\lambda_b D &\approx T n^2 \\
\lambda_b &\approx \frac{T n^2}{D}
\end{aligned} \tag{2.3.4}$$

2.4 Relaxing LFO Assumptions

Equation (2.2.4) is the initial loss function, however it does have some shortcomings, because it is designed whilst making a number of key assumptions. Firstly, it assumes that the contribution from each well is constant over time. This is clearly not true, and is a poor assumption, but it helps us to greatly simplify the expression to create an initial loss function. Next this will be relaxed by introducing linear time-dependence into the system. Instead of setting $w_{i,t} = w_i$, it becomes $w_{i,t} = w_{c,i} + w_{g,i}t$ where $w_{c,i}$ represents the y-intercept of the line representing the extraction rate over time of well i , and $w_{g,i}$ represents its gradient. The new loss function is shown in equation (2.4.1)

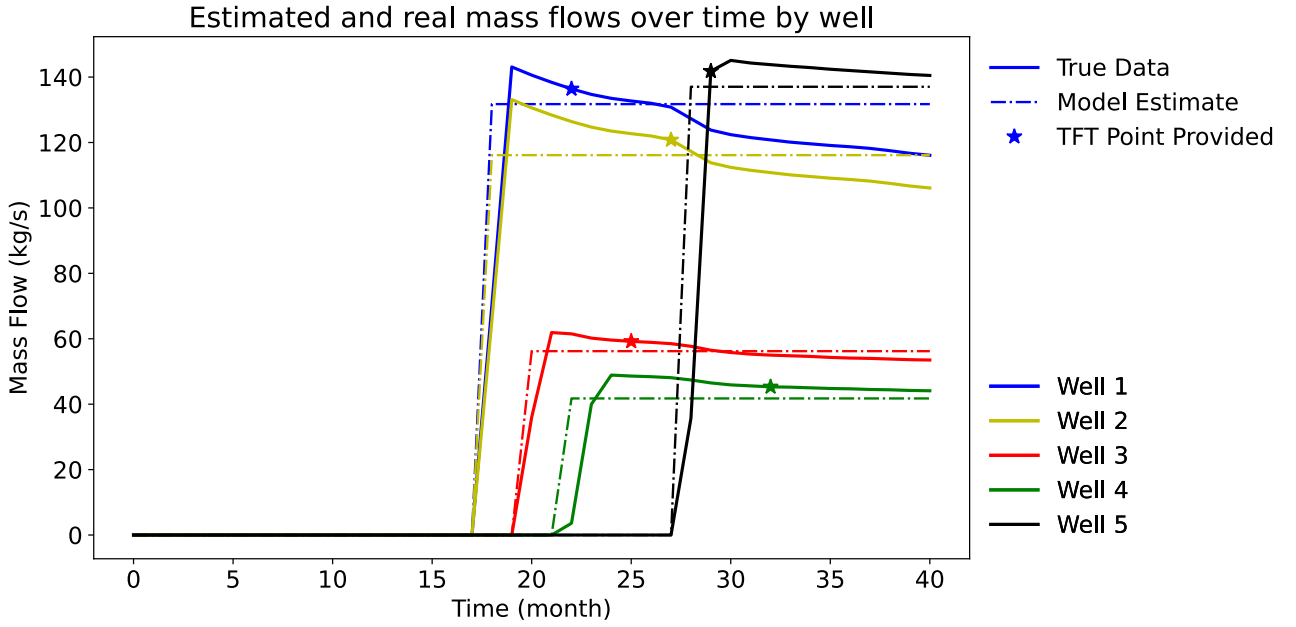


Figure 2.1: Well-specific mass flows predicted by LFO model against real results

$$\mathcal{L}(w_1, w_2 \dots w_n) = \sum_{t=1}^T (M_t - \sum_{i=1}^n z_{i,t}(w_{c,i} + w_{g,i}t))^2 + \lambda \sum_{i=1}^n \sum_{t=1}^T d_{i,t}(m_{i,t} - (w_{c,i} + w_{g,i}t))^2 \quad (2.4.1)$$

This results in a similar matrix equation as in equation (2.3.2), however the w vector will now be twice as long because there are now two coefficients (w_c and w_g) for each well, and therefore $A \in \mathbb{R}^{2n \times 2n}$. This can again be differentiated symbolically and solved to get the following results shown in figure 2.2. These results look similar to the results of the initial model, however because of the linear time-dependence, they are able to more effectively match the given data.

The logical next step is to continue relaxing the assumption of linear time-dependence by creating a quadratic model, cubic, quartic, and so on. To simplify this process, a model was created that can handle an arbitrarily high order polynomial (AHOP). A new parameter k is introduced that represents the highest degree of the polynomial. Equation (2.4.2) shows how this arbitrarily high order polynomial is defined, and equation (??) defines the associated loss function.

$$W_i(t) = w_{i,0} + w_{i,1}t + w_{i,2}t^2 + \dots + w_{i,k}t^k \quad (2.4.2)$$

$$\mathcal{L} = \sum_{t=1}^T (M_t - \sum_{i=1}^n z_{i,t}W_i(t))^2 + \lambda \sum_{i=1}^n \sum_{t=1}^T d_{i,t}(m_{i,t} - W_i(t))^2 \quad (2.4.3)$$

The AHOP model produced interesting results. Results for $k=3$ (cubic) are shown in figure 2.3, which are clearly far worse than the results of both the flat and linear models. This is because with the cubic model having 20 total decision variables, it is able to overfit to the provided data. It is able to ensure predictions meet the TFT points even more accurately than the other models, by introducing wild variation that does not exist in reality. The AHOP model produces even more rapidly varying and inaccurate predictions for higher values of k . To solve this, a regularisation term(s) will be added to equation (2.4.3) with the aim of decreasing the magnitude of the higher order polynomial coefficients.

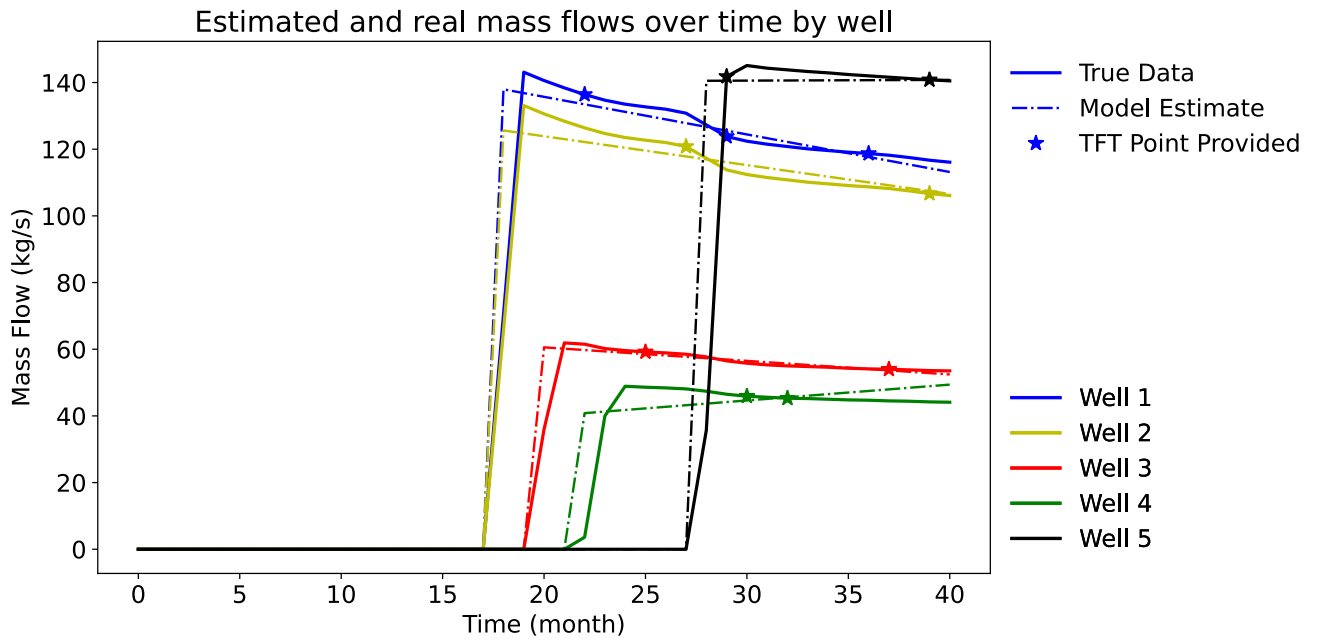


Figure 2.2: Mass flows predicted by linear LFO Model against true mass flows.

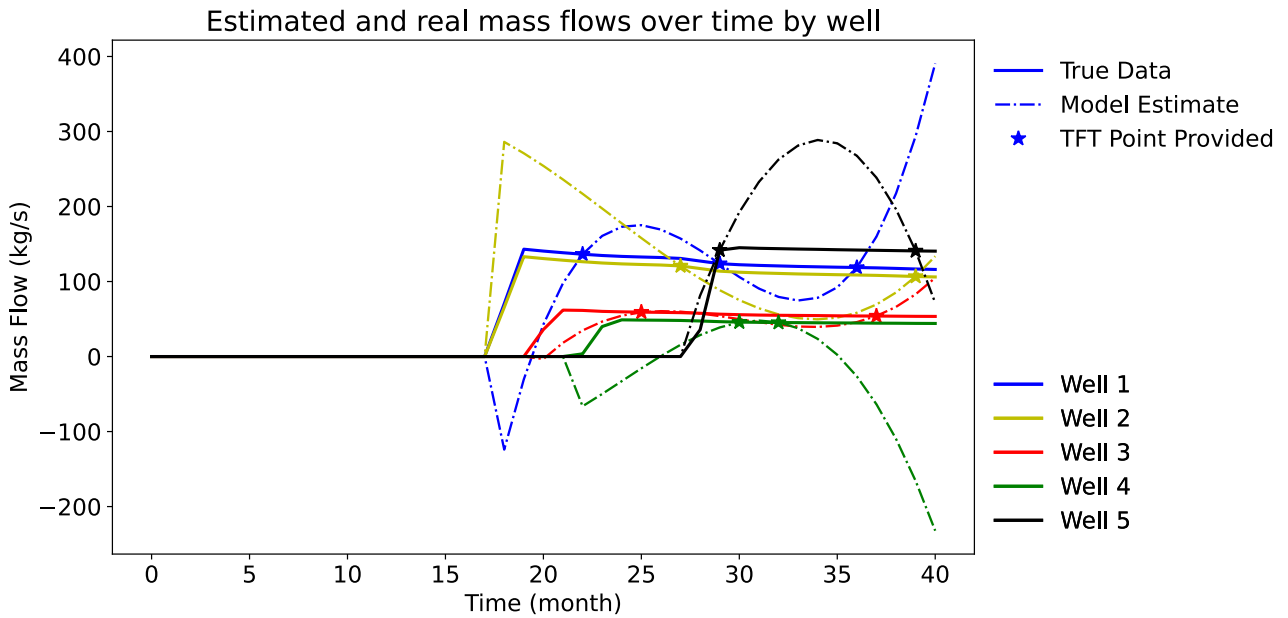


Figure 2.3: Mass flows predicted by AHOP LFO model for $k=3$ (cubic) against true mass flows

2.5 Applying Tikhonov Regularisation to Reduce Over-fitting

There is a number of ways Tikhonov regularisation could be implemented to improve my model fit to this data. The first way attempted was by adding an L_2^2 norm (Ridge regression) term to equation (2.4.3), as shown in equation (2.5.1). This did reduce the size of the coefficients as expected, but because the sum squared error is being used, and the lower order polynomial coefficients tend to have a much larger magnitude, it much more heavily penalised the lower order coefficients (such as the constant and linear terms) than the higher order terms (such as the quadratic and cubic coefficient), which did somewhat smooth out the results as shown in (2.4). However, it also artificially forced the y-intercept of these polynomials towards 0, which is not a realistic constraint to add to the model. For this reason, the ridge regression term was removed from the model, and replaced with a new form of Tikhonov regularisation.

$$\mathcal{L}_{new} = \mathcal{L}_{old} + \lambda_2 \sum_{i=1}^n \sum_{j=0}^k w_{i,j} \quad (2.5.1)$$

To ensure the regularisation term 'flattened' the results without artificially decreasing the constant term towards 0, Tikhonov regularisation was conducted on the first and second time-derivatives of $w_i(t)$. This proved to be much more effective at smoothing out the model results. Equation (2.5.2) shows how this regularisation can be applied to the LFO model.

$$\mathcal{L}_{new} = \mathcal{L}_{old} + \lambda_2 * \sum_{i=1}^n \sum_{j=1}^T \left(\frac{dW_i(t)}{dt} \Big|_{t=j} \right)^2 + \lambda_3 * \sum_{i=1}^n \sum_{j=1}^T \left(\frac{d^2W_i(t)}{dt^2} \Big|_{t=j} \right)^2 \quad (2.5.2)$$

$$7 \quad (2.5.3)$$

The next logical step was to conduct a grid or metaheuristic search across the parameter space of $\lambda_1, \lambda_2, \lambda_3$. Unfortunately the available computation power proved too low to find effect results using this method. Instead, the problem was re-formulated as an inverse problem, where the same loss function as in equation (2.5.3) is expressed as an ill-posed parameter estimation problem. Chapter 3 explored this process in detail.

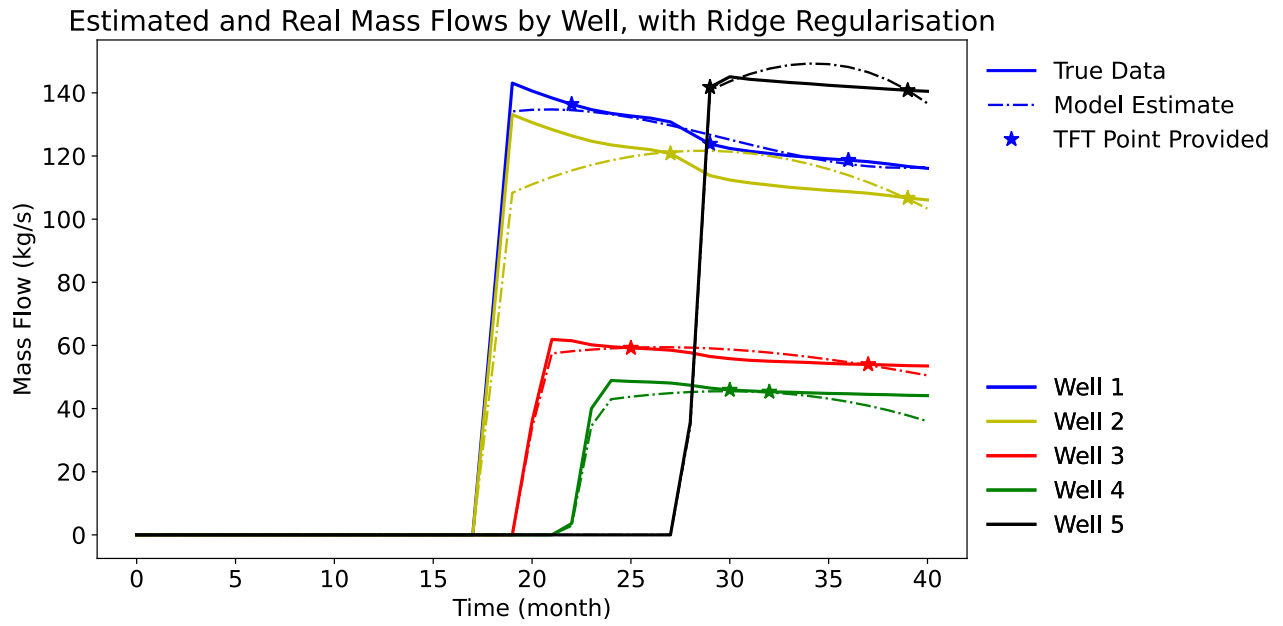


Figure 2.4: Mass flows predicted by AHOP LFO model with Ridge Regularisation for $k=3$ (cubic) against true mass flows

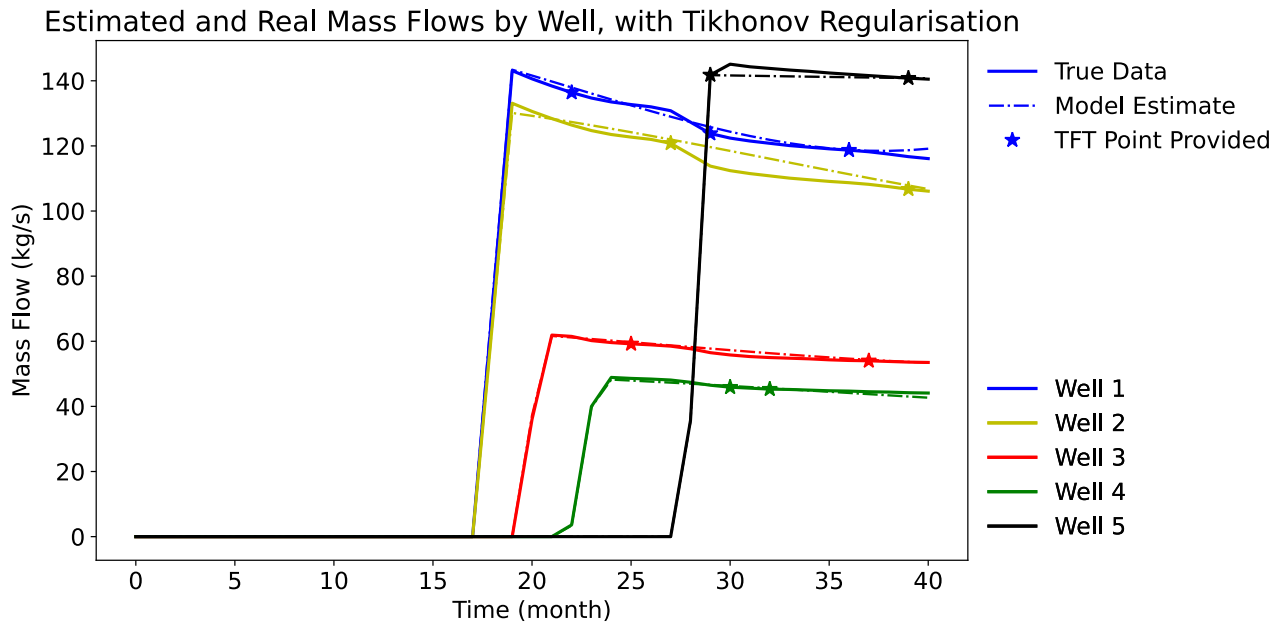


Figure 2.5: Mass flows predicted by AHOP LFO model with Tikhonov Regularisation for $k=3$ (cubic) against true mass flows

Chapter 3

Linear Least Squares Optimisation (LLSO)

3.1 Reformulating the Problem as an Inverse Problem

The general form of an inverse problem is shown in equation (3.1.1). The A matrix is then decomposed into A_1 and A_2 , where A_1 contains measured data such as when the wells were on/off, and A_2 contains the chosen model information, as shown in equation (3.1.2). Note that $k \ll T$, so the y vector is much longer than the x vector, meaning the problem is overconstrained. This is what allows it to be solved in a least squared fashion to find the 'best fit' to the data by minimising squared error. An example of this matrix formulation for arbitrary values of n , T and k is shown in equation (3.1.3). This shows how the existing problem data labelled M_t , $m_{i,t}$, $z_{i,t}$ and $d_{i,t}$ (As defined in equation (2.2.1) and (2.2.2)) can be input into the LLSO model.

$$\begin{aligned} y &= Ax + e \\ y &\approx Ax \end{aligned} \tag{3.1.1}$$

where:

$y \in \mathbb{R}^{T(n+1)}$ contains individual and total mass data.
 $A \in \mathbb{R}^{T(n+1) \times n(k+1)}$ contains model information and other measured data
 $x \in \mathbb{R}^{n(k+1)}$ is our vector of polynomial coefficients
 $e \in \mathbb{R}^{T(n+1)}$ is the error / noise

$$\begin{aligned} A &= A_1 A_2 \\ y &\approx A_1 A_2 x \\ y &\approx A_1 w \\ w &= A_2 x \end{aligned} \tag{3.1.2}$$

where:

$A_1 \in \mathbb{R}^{T(n+1) \times Tn}$ contains measurement and on/off well data
 $A_2 \in \mathbb{R}^{Tn \times n(k+1)}$ contains our chosen model
 $w \in \mathbb{R}^{Tn}$ is our vector of w values for all wells at all times

$$\begin{bmatrix} m_{11} \\ m_{12} \\ \vdots \\ m_{1T} \\ m_{21} \\ \vdots \\ m_{nT} \\ M_0 \\ M_1 \\ M_2 \\ \vdots \\ M_T \end{bmatrix} = \begin{bmatrix} d_{11} & 0 & \dots & 0 & 0 & \dots & \dots & 0 \\ 0 & d_{12} & & \vdots & \vdots & & & \vdots \\ \vdots & & \ddots & \vdots & \vdots & & & \vdots \\ 0 & \dots & \dots & d_{1T} & \vdots & & & \vdots \\ 0 & \dots & \dots & \dots & d_{21} & & & \vdots \\ \vdots & & & & & \ddots & & \vdots \\ \vdots & & & & & & \ddots & \vdots \\ 0 & \dots & \dots & \dots & \dots & \dots & \dots & d_{nT} \\ z_{11} & 0 & \dots & 0 & z_{21} & & & \vdots \\ 0 & z_{12} & & \vdots & & z_{22} & & \vdots \\ \vdots & & \ddots & \vdots & & & \ddots & \vdots \\ 0 & \dots & \dots & z_{1T} & & & & z_{nT} \end{bmatrix} \begin{bmatrix} 1 & t_1 & t_1^2 & \dots & t_1^k & 0 & \dots & \dots & \dots & 0 \\ 1 & t_2 & t_2^2 & \dots & t_2^k & \vdots & & & & \vdots \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & & & & \vdots \\ 1 & t_T & t_T^2 & \dots & t_T^k & 0 & \dots & \dots & \dots & 0 \\ 0 & \dots & \dots & \dots & 0 & 1 & t_1 & t_1^2 & \dots & t_1^k \\ \vdots & & & & \vdots & 1 & t_2 & t_2^2 & \dots & t_2^k \\ \vdots & & & & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & \dots & \dots & 0 & 1 & t_T & t_T^2 & \dots & t_T^k \end{bmatrix} \begin{bmatrix} x_{10} \\ x_{11} \\ x_{12} \\ \vdots \\ x_{1k} \\ x_{20} \\ x_{21} \\ x_{22} \\ \vdots \\ x_{nk} \end{bmatrix} \quad (3.1.3)$$

This does not yet include any regularisation because the model is being kept as simple as possible. This is so that the results of the initial LLSO Model can be easily compared to the results of the equivalent LFO Model. As shown in figure (3.1), results of the two models were nearly identical. This shows that the model is functioning as expected.

3.2 Speed Comparison of LFO and LLSO Models

This made the process of solving my model much more efficient. Upon comparison of the speed of each method, using the Python 'time' package, the LFO Model took about 41 seconds to solve, meanwhile the LLSO Model took only about 3 seconds, indicating a speedup of around 1250%. Seedup is defined as the percentage improvement of time taken to solve the LLSO Model, relative to the LFO Model, given the same inputs. This is evaluated this for a number of different values of T and k, to see if the speedup is dependent on these values. Results are shown in table (3.1). For all values examined, there is significant speedup. It is 12x faster for small problems ($T = 6, k = 0$), up to 1000x faster for large problems ($T = 41, k = 5$). For real-world problems the values for T and k will be even higher, so the speedup will be even more significant.

Interestingly, the only difference between the T=23 and T=41 examples are rows made entirely out of 0s, when all wells were turned off. While this does not provide any more

$$\begin{array}{ll}
\text{LFO:} & \begin{bmatrix} 0.0755281423089116t^2 - 5.64731044978907t + 224.078448886958 \\ 0.00103016571017633t^2 - 1.24505839921767t + 153.667290010839 \\ 0.0115292700365899t^2 - 1.15147522214102t + 80.7917101942003 \\ 0.0207668544503576t^2 - 1.74062711643637t + 79.5758440514328 \\ 0.0473494068028231t^2 - 3.31198527943343t + 197.934526901692 \end{bmatrix} \\
\text{LLSO:} & \begin{bmatrix} 0.0755281423081833t^2 - 5.79836673440658t + 229.801287480164 \\ 0.0010301657116872t^2 - 1.24711873073357t + 154.913378577739 \\ 0.0115292700332608t^2 - 1.17453376199814t + 81.9547146794474 \\ 0.0207668544464444t^2 - 1.78216082500748t + 81.3372380172163 \\ 0.047349406820303t^2 - 3.40668409424758t + 201.293861615931 \end{bmatrix}
\end{array}$$

Figure 3.1: Equations $W_i(t)$ for each well, when solved for k=2 by LFO and LLSO Models

Table 3.1: Speedup of new method compared to old method for different T and k values, with fixed n=5, rounded to nearest 100%.

<i>speedup</i> [%]	k=0	k=1	k=2	k=3	k=4	k=5
T=6	1200	2400	6500	9700	17000	10400
T=11	2500	6000	10700	22700	28900	36800
T=17	3500	11100	22300	31800	45700	62500
T=23	7500	14900	26600	37800	55400	65800
T=41	21300	29600	43400	60600	79200	103500

information to the model, it does slow the system down in finding the optimal solution (for both LFO and LLSO models). This means to increase efficiency even further, rows should be removed from the y vector and A matrix if they contain only zeroes in both, before solving the model. Because only a small percentage of m_{it} values are known (corresponding to points when $d_{it} = 1$), this will greatly reduce the size of the A array. Theoretically, this row-reduced form should be much smaller making it much faster to solve. At the small scales of the provided simulated data, there did not appear to be a significant speedup from making this change, because of the computational cost associated with doing these deletions. However for higher scale problems, this will be more efficient. The results produced by the original and row-reduced methods are identical (up to negligible machine error).

3.3 Tikhonov Regularisation for LLSO

Next, Tikhonov regularisation is applied to the LLSO Model. More rows need to be added to A_1 and y to achieve this regularisation, making the matrix approximately three times taller. The rows attached to A_1 will contain n finite-difference stencil blocks along the diagonal to numerically differentiate $w = A_2x$. This stencil for first order Tikhonov regularisation is shown in equation (3.3.1), and the second order equivalent is shown in (3.3.2). Similarly, an appropriately-sized 0 vector is attached to y , as the goal is to minimise these derivatives.

$$A_{Tik,1} = \sqrt{\lambda_2} \begin{bmatrix} -1 & 1 & 0 & \dots & 0 & 0 & \dots & \dots & \dots & 0 \\ 0 & -1 & 1 & 0 & \vdots & \vdots & & & & \vdots \\ \vdots & & \ddots & \ddots & \vdots & \vdots & & & & \vdots \\ 0 & \dots & 0 & -1 & 1 & 0 & \dots & \dots & \dots & 0 \\ 0 & \dots & \dots & \dots & 0 & -1 & 1 & 0 & \dots & 0 \\ \vdots & & & & \vdots & 0 & -1 & 1 & 0 & \vdots \\ \vdots & & & & \vdots & & & \ddots & \ddots & \vdots \\ 0 & \dots & \dots & \dots & 0 & 0 & \dots & 0 & -1 & 1 \end{bmatrix} \in \mathbb{R}^{n(T-1) \times nT} \quad (3.3.1)$$

$$A_{Tik,2} = \sqrt{\lambda_3} \begin{bmatrix} 1 & -2 & 1 & \dots & \dots & 0 & 0 & \dots & \dots & \dots & \dots & 0 \\ 0 & 1 & -2 & 1 & & \vdots & \vdots & & & & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & \vdots & \vdots & & & & & \vdots \\ 0 & \dots & \dots & 1 & -2 & 1 & 0 & \dots & \dots & \dots & \dots & 0 \\ 0 & \dots & \dots & \dots & \dots & 0 & 1 & -2 & 1 & \dots & \dots & 0 \\ \vdots & & & & & \vdots & 0 & 1 & -2 & 1 & & \vdots \\ \vdots & & & & & \vdots & & & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & \dots & \dots & \dots & 0 & 0 & \dots & 0 & 1 & -2 & 1 \end{bmatrix} \in \mathbb{R}^{n(T-2) \times nT} \quad (3.3.2)$$

These newly formed matrices $A_{Tikh,1}$ and $A_{Tikh,2}$ can be attached to our original A_1 matrix as shown in equation (3.3.3) to give us our new $A_{1,new}$ matrix that can be used in conjunction with y_{new} to solve in a least squares fashion using Tikhonov regularisation. Replacing y with y_{Tik} and A with A_{Tik} in equation (3.1.1) gives us our final least squares equation, shown in equation (3.3.4)

$$\begin{aligned}
A_{1,Tik} &= \begin{bmatrix} A_1 \\ A_{Tikh,1} \\ A_{Tikh,2} \end{bmatrix} \in \mathbb{R}^{T(n+1)+n(T-1)+n(T-2) \times nT} \\
y_{Tik} &= \begin{bmatrix} y \\ 0 \\ 0 \end{bmatrix} \in \mathbb{R}^{T(n+1)+n(T-1)+n(T-2)} \\
A_{Tik} &= A_{1,Tik} A_2 \in \mathbb{R}^{T(n+1)+n(T-1)+n(T-2) \times n(k+1)}
\end{aligned} \tag{3.3.3}$$

$$y_{Tik} \approx A_{Tik} x \tag{3.3.4}$$

There are now 3 parameter that need to be tuned to get the best quality results from the LLSO model. To quantify the results, Root Mean Square Error (RMSE) was chosen. This was calculated across all wells and timepoints, as even the provided TFT points will not be met perfectly by the LLSO model. The parameter space $\mathbf{\Lambda} = [\lambda_1, \lambda_2, \lambda_3]$ where λ_1 represents the weighting put on the TFT points, λ_2 represents the weighting of the first derivative Tikhonov regularisation and λ_3 represents the weighting of the second derivative Tikhonov regularisation.

To conduct parameter tuning, a particle swarm optimisation metaheuristic was used. This metaheuristic is not guaranteed to give the optimal parameter values, but it will give good results in a reasonable amount of time and can be easily applied to a black-box function such as the LLSO model. A fixed value of $k = 5$ was used. The $\hat{\mathbf{\Lambda}}$ that gave the lowest RMSE for the simulated dataset was $\hat{\lambda}_1 = 135.13$, $\hat{\lambda}_2 = 7.52$ and $\hat{\lambda}_3 = 0$. This resulted in a RMSE of 6.50 kg/s. It is interesting that λ_3 has an 'optimal' value of 0. This indicates that second derivative Tikhonov regularisation is unnecessary, and that first derivative Tikhonov regularisation is more effective. In chapter 4, these optimal values $\hat{\mathbf{\Lambda}}$ will be used to produce final results, which will be discussed in detail in chapters 4 and 5.

Chapter 4

Results with Simulated and Real Data

4.1 Quantitative Results of LLSO Model on Simulated and real Data

The best results found from parameter tuning of the LLSO Model are shown in figure 4.1. This is clearly the best result yet, as the model nearly perfectly matches the true data. It is not overfit or underfit. The RMSE of 6.50 is relatively small. This indicates that for simple scenarios such as this simulated dataset, the LLSO model is an effective tool for inferring the extraction history of a geothermal field. However real datasets are likely to be much more messy, with missing data and measurement errors meaning that data will never exactly add up.

Thanks to Paul from Contact Energy, real data taken from a real geothermal field somewhere in New Zealand was provided for testing. Because this is real (not simulated) data, the true values for mass flow are not known. However, an expert's guess as to how much mass was extracted from each well during each day has been provided. This is not necessarily an accurate measurement, but it is currently the best guess that exists, so it will be helpful to compare the model's results against this estimate. The TFT points are now separate from the 'true' data as that no longer exists, so have to be provided to the model separately. Equation (??) is modified so that the m matrix now represents the provided TFT points, and the d matrix now represents whether or not a TFT point was or wasn't provided to the model. Similarly, the z matrix of when wells were on or off are now determined not from the 'true' data, but instead from the expert's estimate.

We (Michael and myself) worked together to manipulate the real data such that it could be inputted to our models. This was mainly achieved by simply manipulating the data within an Excel spreadsheet, however the provided TFT points required more intense data analysis before it was possible to pick out which points should be used where within the model. For this, an R script was used to analyse and manipulate the TFT data from the provided Excel sheet, removing missing values and appropriately matching the points to the correct day of measurement. From the available data, separator 3 for a given time-period was chosen to test our models, because having only 3 active wells, and a timeframe of around 1000 days produced clearer plots that could easily be compared to each other's and the expert's estimate. The results produced by the LLSO Model are shown in 4.2

The model estimates shown in Figure 4.2 somewhat aligns with the expert's estimates. For Well 1, which has many TFT points, the model estimate appears to be a better estimate than the expert's estimate because it more closely matches the provided TFT points. For well 2, The estimates also match fairly closely throughout the full 1461 days. At certain points, the LLSO model under-predicts (relative to the expert's estimate), and at other points is over-predicts, so overall predicts a similar amount of extraction. For well 3, the estimates align closely through the first 800 days when there are many TFT points, but diverge after this point, when there are less TFT points provided. It is difficult to tell

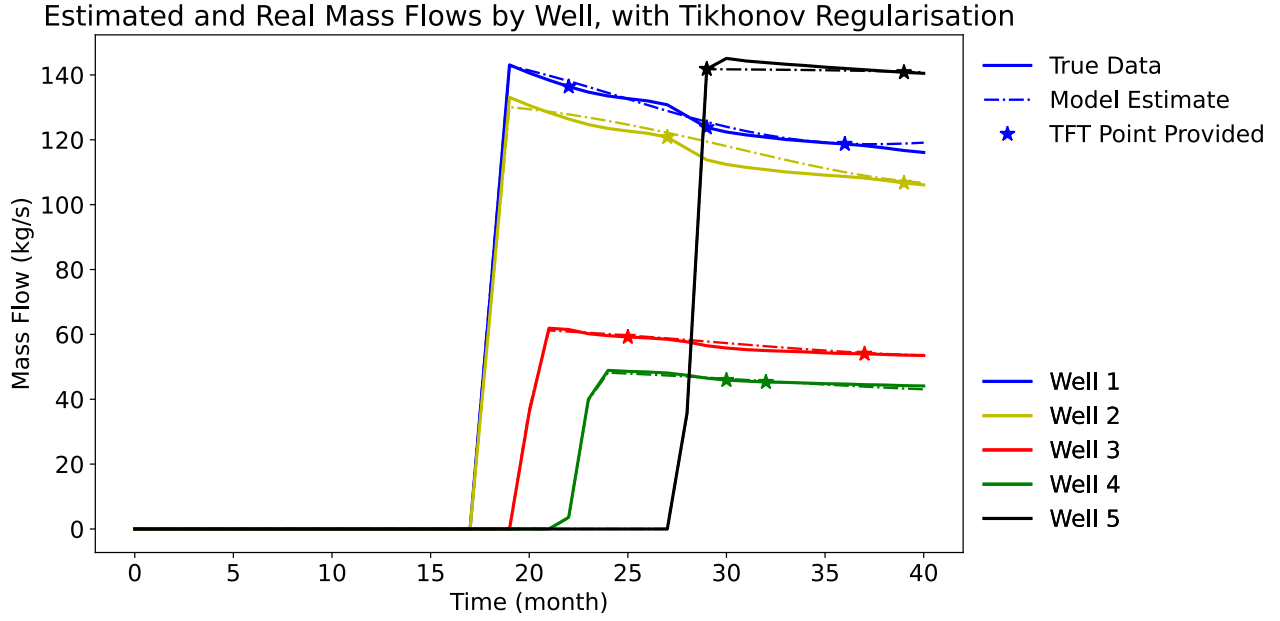


Figure 4.1: Mass flows predicted by LLSO model for simulated data, for $k = 4$ and optimal parameter values.

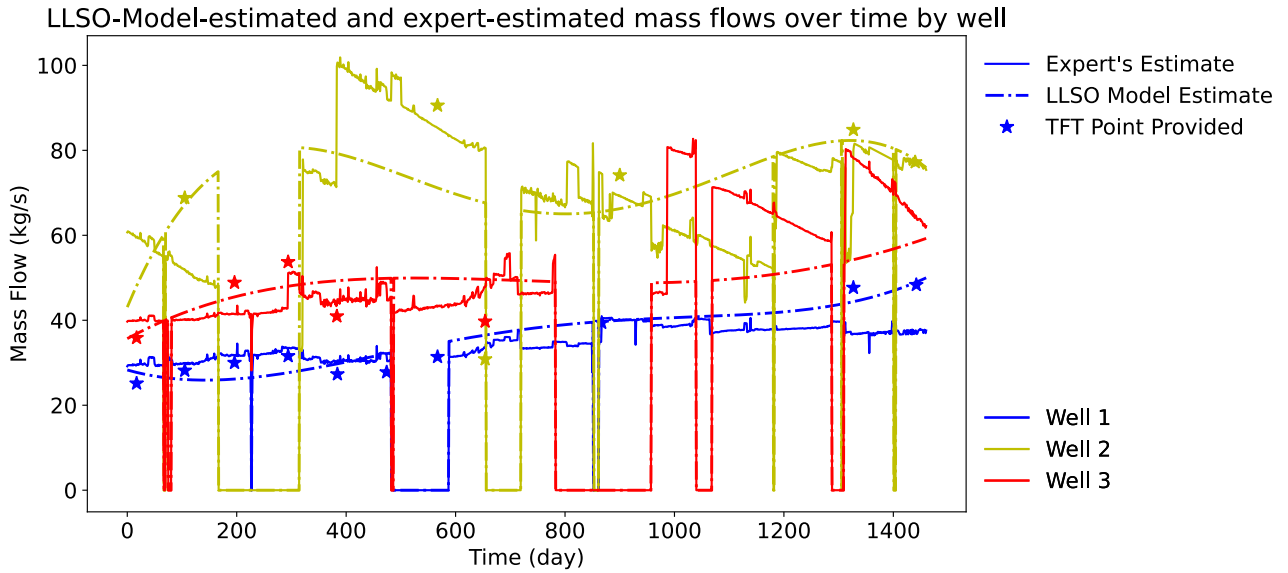


Figure 4.2: Mass flows predicted by LLSO model for real data, for $k = 4$ and optimal parameter values.

which estimate is more accurate in this region. RMSE cannot be computed against the true extraction from this well as it is not known, but it can be computed against the expert’s estimate. However, this would be a poor measurement of the accuracy of the model as the expert’s estimate could be at least as wrong as the model’s estimate.

4.2 Quantifying LLSO Model Uncertainty

To quantify the uncertainty of the final model, Monte Carlo simulations were used. Gaussian noise with a mean of 0 and a standard deviation of 20 kg/s was added to the known TFT points, to represent the uncertainty in the provided data. 2,000 models were then run, each getting a different output because of the added noise. This then allows the variance of the output model to be quantified. The standard deviation was calculated for each timepoint for each well, and was found to be in the range of — to — kg/s. This was larger (??) than the inputted standard deviation, so some additional uncertainty is being added by the LLSO Model, as expected.

The 25th to 75th percentile of predictions are shown in figure 4.3. This shows how the uncertainty is lower near TFT points, but increases when further from TFT points. Similarly, the uncertainty increases near the edges of the time-domain when the model is effectively extrapolating instead of interpolating. While the LLSO model does not provide highly certain estimates, it is important that uncertainty can be quantified, so that geothermal modelers can input this uncertainty into predictive models.

4.3 Comparison with other models

Michael’s Gaussian Process Regression (GPR) Model produces quite different results from my LLSO model. These are shown in Figure 4.4. Michael’s results have more variability in predictions, and allow for sharp changes in mass flows for each well, whereas LLSO enforces smoothness, and therefore gives a more consistent estimate. Michael’s model seems to predict less variability, as the 95% confidence intervals shown converge tightly around known TFT points, for both the known well and the other wells. This is because the GPR enforces each sample draw to go exactly through each TFT point, with no uncertainty applied to that measurement. This is an interesting approach, however it can be considered to overfit to the data in this way because this certainty results in a highly variable prediction for each well over time, with sharp increases and decreases linking the known TFT points.

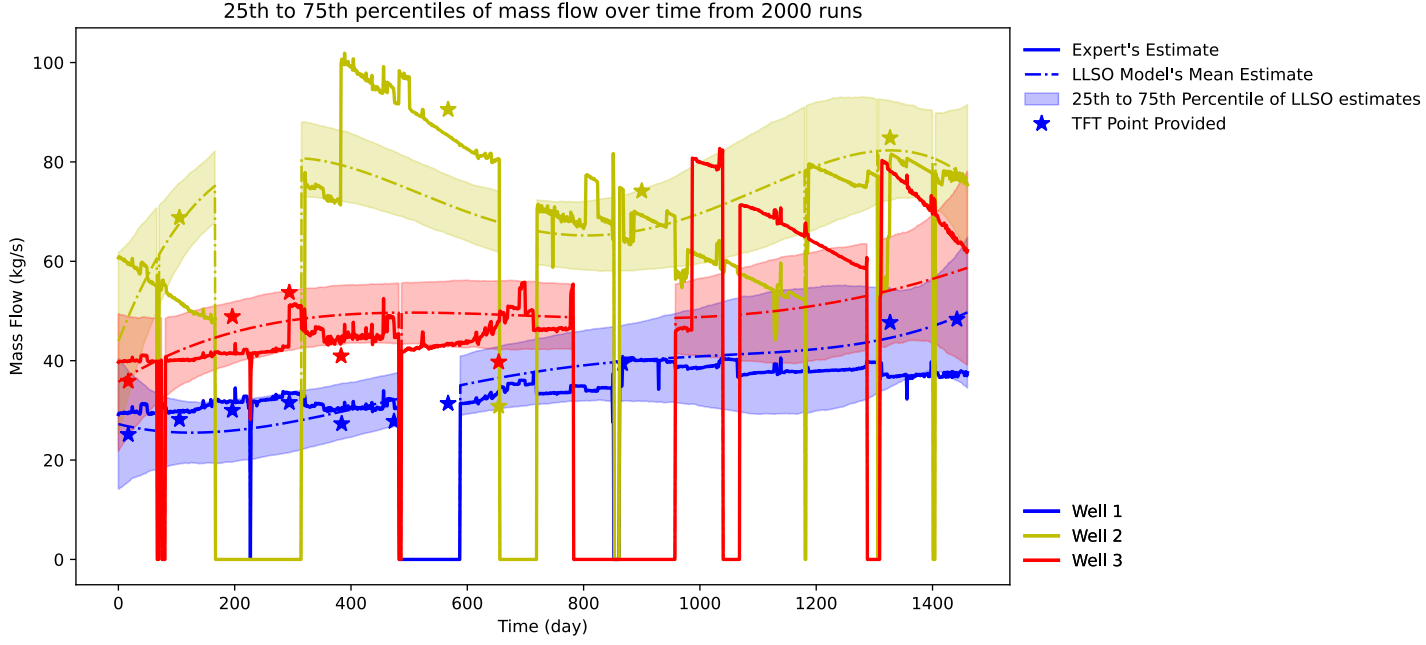


Figure 4.3: 25th to 75th percentiles of mass flows predicted by LLSO model for real data, for $k = 4$ and optimal parameter values.

Applying GPR to impute missing mass flows for multiple wells for 1461 days using 95% confidence intervals

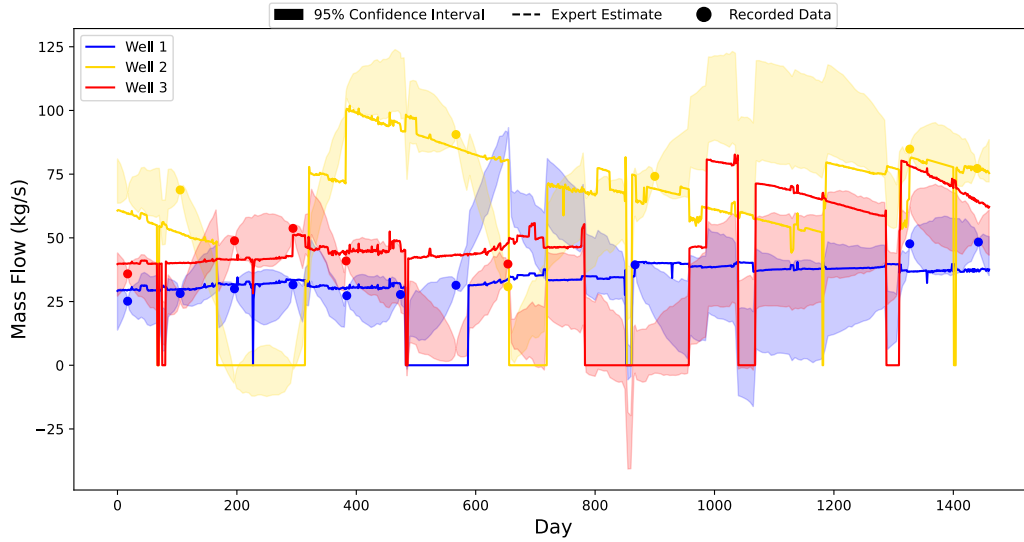


Figure 4.4: 95% confidence intervals of Michael's GPR Model for real data.

Chapter 5

Discussion

5.1 Discussion of LLSO and GPR Model Effectiveness

As a first attempt at predicting the data history of a geothermal field using data-driven methods, the LLSO and GPR models both succeed. Visually, the deterministic LLSO model appears to provide a better prediction for the data history of each well as it is smoother and more consistent. Meanwhile, the Bayesian GPR model produces results with more easily quantifiable uncertainty, and produces an interval of predictions instead of a single prediction, which may be helpful for geothermal modelers.

By comparing Figures 4.3 and 4.4, we can see that the LLSO model (using Monte Carlo simulations) predicts much more uncertainty than the GPR model. This is likely because the LLSO predictions are not forced to exactly meet each TFT point, while the GPR predictions are. With contextual knowledge of the uncertainty associated with these data points, we could determine which approach makes more sense. If there is high confidence in these measurements then the GPR approach make more sense, but if there is low confidence in these measurements (which is more likely for this specific dataset) the LLSO approach seems more realistic.

The enforced smoothness (due to Tikhonov regularisation) of the LLSO model makes it produce more realistic seeming results than the GPR model. For example, for well 1 the LLSO predicts a slowly increasing trend across time, meanwhile the GPR model predicts a highly varying extraction which goes from around 25 kg/s to around 90 kg/s within only 50 days. Also, the LLSO model appears to be more realistic because (while nothing explicitly prevents it from doing this) it never predicts any negative mass flows. Contrastingly, the GPR model predicts a negative mass flow at least once for each well. However, this enforced smoothness can also be over-restricting, resulting in LLSO struggling to model the finer behaviour of geothermal wells over time. This may not be a large issue, as geothermal modelers will be more concerned with the big-picture of the mass extraction, as opposed to the small variations that naturally occur over time. However, this is still a drawback of the LLSO model.

The LLSO model predictions for a given well also appear to be more independent of other well predictions. For example, when LLSO predicts a temporary increase in the extraction of well 2, we do not see a corresponding decrease in wells 1 and 3. However, this is what happens for the GPR model, because it enforces the three well measurements to add exactly up to the total extraction at each point. The LLSO model allows there to be some error here, which enables more independence between wells. This could be considered a strength or weakness, depending on whether this is expected behaviour for a given geothermal field. For example, if the wellhead pressures are adjusted so that when one well produces less, another will take its place and produce more, then the GPR model results make sense. However, if one well decreases its output and there is no change to the other wells then the LLSO model results would model this more accurately.

The accuracy of both models is limited by the amount and certainty of data that is

provided to them. The rarity of TFT measurements, combined with their high uncertainty (at least for this dataset) are the main thing preventing the LLSO and GPR models from making better and more certain predictions. There are TFT measurements in the provided real dataset which are clearly wrong. Some values are higher than the total extraction mass on that day, which is impossible. Many other datapoints have notes from a reservoir engineer saying that the flow was too high, too low, or otherwise seemed wrong. The separator 3 data shown in Figures 4.3 and 4.4 was one of the cleanest parts of the overall dataset because it has reasonable TFT measurements and enough of them to make decent predictions.

5.2 Conclusions and Future Work

Overall, both the LLSO and GPR models are a vast improvement over trying to estimate the extraction history manually. The LLSO provides a reliable algorithm which is efficient enough to handle large datasets with ease. Importantly, using the LLSO model removes the subjectivity of the manual process, and instead provides a robust and objective method for inferring the extraction history of a geothermal field. It has developed a lot over the course of this project but there is still room for improvement through future work. In particular, there are three suggestions below for specific next steps that would improve the performance of the LLSO model even further:

The most significant thing constraining the model (other than the rarity and uncertainty of data) is that it can only model the mass extraction history using polynomials. Even for high degree polynomials this restricts the behaviours that can be modeled by LLSO. This could be improved in a number of ways. Trigonometric terms such as $\sin(t)$ or $\sin(\frac{1}{365}t)$ could be added to the polynomial to incorporate periodic activity if there is known seasonal variation at a given geothermal field. Alternatively, the polynomial could be broken up into a number of splines which combine to form a continuous function across the whole time domain. This would also enable more sharp changes to occur as opposed to a single polynomial which can only change smoothly across time.

Non-negativity should also be enforced for the LLSO model. While it has not made any negative predictions thus far, there is nothing preventing the model from making a negative mass flow prediction. This would be a large issue as this is clearly not possible. To fix this, LLSO could be performed iteratively. If it predicts any negativity, an artificial TFT point (or multiple) could be added to the data to 'pull' the prediction in the positive direction. This would be repeated until the model is making only non-negative predictions.

The confidence in TFT values is often not constant between data points. This could be reflected by including weightings on TFT values which reflect the confidence in their accuracy. Weightings of 1 would indicate an average confidence in that point, while > 1 indicates high confidence and < 1 indicates low confidence. These would give higher importance to our model matching more certain TFT measurements, and would reduce the importance of matching the prediction to less certain measurements.

In conclusion, the LLSO model successfully provides an objective and fast alternative to manually estimating the extraction history of a geothermal field. It makes reasonable predictions on both simulated and real data, and its uncertainty can be quantified using Monte Carlo simulation. However, it is limited by the quantity and accuracy of TFT points that it is provided, and can struggle to model the finer behaviour of the geothermal field because it only uses a single polynomial to model the whole extraction history for each well. Depending on the data provided, and the use case of the outputted data history, either the LLSO or GPR model may be preferred over the other, but for the real data provided the LLSO appears to model the extraction history more accurately. There are a number of next steps that can be taken to improve the LLSO model, including enforcing non-negativity, using more complicated non-polynomial forward models, and individually weighting TFT points. If these changes are made this method has the potential to become applicable across many geothermal fields across the world.

Chapter 6

Bibliography

References

- [1] Nz geothermal fields, <https://www.geothermal.auckland.ac.nz/en.html> (2024).
- [2] J. H. S. Wendell A Duffield, Geothermal Energy, Clean Power from the Earth's Heat, U.S. Department of the Interior, 2003.
- [3] G. U.-b. Janusz A. Kubiak a, Simulation of the effect of scale deposition on a geothermal turbine, Geothermics (2002).
- [4] L. Rybach, Geothermal sustainability, GHC Bulletin (2007).
- [5] M. P. Hochstein, Assessment and modelling of geothermal reservoirs (small utilization schemes), Geothermics (1988).
- [6] R. van de Schoot et al., Bayesian statistics and modelling, Nature Reviews Methods Primers (2021).
- [7] J. K. C. Noah D. Athens, A monte carlo-based framework for assessing the value of information and development risk in geothermal exploration, Applied Energy (2019).
- [8] A. B. Downey, Think Bayes, Green Tea Press, 2012.
- [9] P. L. Bonate, A brief introduction to monte carlo simulation, Clinical Pharmacokinetics (2012).
- [10] T. G. Dietterich, Ensemble methods in machine learning, in: Multiple Classifier Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2000, pp. 1–15.
- [11] K. Csilléry, M. G. Blum, O. E. Gaggiotti, O. François, Approximate bayesian computation (abc) in practice, Trends in Ecology I& Evolution 25 (7) (2010) 410–418. doi:10.1016/j.tree.2010.04.001.
URL <https://www.sciencedirect.com/science/article/pii/S0169534710000662>
- [12] M. Sadegh, J. A. Vrugt, Approximate bayesian computation using markov chain monte carlo simulation: Dream(abc), Water Resources Research 50 (8) (2014) 6767–6787. arXiv:<https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1002/2014WR015386>, doi:10.1002/2014WR015386.

URL <https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1002/2014WR015386>

- [13] C. Barnes, S. Filippi, M. e. a. Stumpf, Considerate approaches to constructing summary statistics for abc model selection, *Stat Comput* 22 (2012).
- [14] A. Bharti, Approximate bayesian computation with domain expert in the loop, <https://www.youtube.com/watch?v=Gx18PH0kiY8> (2023).
- [15] X. Shi, D. Jiang, W. Qian, Y. Liang, Application of the gaussian process regression method based on a combined kernel function in engine performance prediction, *ACS Omega* (2022).
- [16] M. I. Jordan, T. M. Mitchell, Machine learning: Trends, perspectives, and prospects, *Science* 349 (6245) (2015) 255–260. arXiv:<https://www.science.org/doi/pdf/10.1126/science.aaa8415>, doi:10.1126/science.aaa8415. URL <https://www.science.org/doi/abs/10.1126/science.aaa8415>
- [17] Q. Wang, Y. Ma, K. e. a. Zhao, A comprehensive survey of loss functions in machine learning, *Annals of Data Science* (2022).
- [18] E. R. O. et al., Machine learning in subsurface geothermal energy: Two decades in review, *Geothermics* (2022).
- [19] P. M. B. Abrasaldo, S. J. Zarrouk, A. W. Kempa-Liehr, A systematic review of data analytics applications in above-ground geothermal energy operations, *Renewable and Sustainable Energy Reviews* 189 (2024) 113998. doi:10.1016/j.rser.2023.113998. URL <https://www.sciencedirect.com/science/article/pii/S1364032123008560>
- [20] M. R. Segal, Machine learning benchmarks and random forest regression, UCSF: Center for Bioinformatics and Molecular Biostatistics (2004).
- [21] J. Li, L. Qiu, B. Tang, D. Chen, D. Zhao, R. Yan, Insufficient data can also rock! learning to converse using smaller data with augmentation, *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (01) (2019) 6698–6705. doi:10.1609/aaai.v33i01.33016698. URL <https://ojs.aaai.org/index.php/AAAI/article/view/4641>
- [22] K. Poulinakis, D. Drikakis, I. W. Kokkinakis, S. M. Spottswood, Machine-learning methods on noisy and sparse data, *Mathematics* (2023).
- [23] S. Cai, Z. Mao, Z. e. a. Wang, Physics-informed neural networks (pinns) for fluid mechanics: a review, *Acta Mechanica Sinica* (2021).
- [24] P. Banerjee, How ai and ml are changing simulation, *Ansys Blog* (2021).

Chapter 7

Appendices